

Internal Communication & Collaboration System

Architecture Document

1. System Overview

A room-based communication and ticketing system designed to manage internal workflows across a distributed organization (~200 employees). The system is fundamentally different from CRM—it focuses on **communication and task management**, not data storage or customer relationships.

Core value: Scoped visibility, transparent workflows, and asynchronous communication through threaded messages.

2. Organization Structure

2.1 Organizational Hierarchy

```
Organization (~200 employees)
├── Founder Room (1)
│   └── Strategic, company-wide decisions
├── Branch Rooms (3 branches)
│   ├── Branch A
│   ├── Branch B
│   └── Branch C
└── Department Rooms (5-6 departments)
    ├── HR
    ├── Finance
    ├── Operations
    ├── Sales
    ├── Engineering
    └── [Additional departments]
```

2.2 Room Types

Founder Room

- Access: Founders/executives only
- Purpose: Strategic initiatives, company policy, high-level decisions

- Visibility: Not visible to general employees

Branch Rooms

- Access: Employees assigned to that branch
- Purpose: Branch-specific operations, local coordination
- Visibility: Only employees in that branch can see tickets/messages
- Cardinality: 3 physical branches $\times$ 1 room = 3 rooms

Department Rooms

- Access: Employees in that department (across all branches)
- Purpose: Function-specific work (payroll, recruiting, logistics, etc.)
- Visibility: Only department members can see tickets/messages
- Cardinality: 5-6 departments $\times$ 1 room = 5-6 rooms

Cross-Room Tickets (optional)

- A ticket can be assigned to multiple rooms simultaneously
 - Appears in all assigned rooms' feeds
 - Enables coordination when a task affects multiple teams
-

3. Ticket & Message System

3.1 Ticket Lifecycle

1. **Create:** Any employee creates a ticket in their assigned room
 - Title, description, metadata
 - Attached to exactly one or more rooms
 - Auto-assigned a unique ID
2. **Assign:** Creator or manager assigns the ticket to a specific team/person
 - Specifies which room(s) it belongs to
 - Sets priority, due date
3. **Approve:** Manager/lead reviews and approves
 - Can request changes, ask questions
 - Comments appear as part of the thread
4. **Execute:** Assigned person/team takes action
 - Updates progress in the same thread

- May ask for clarification or escalate

5. **Resolve:** Ticket marked complete

- Final comment summarizes what was done
- Remains visible in the room for reference

3.2 Threading Model

Core principle: Everything is a single, continuous thread.

- When you create a ticket, you write the initial message
- Every comment, approval, question, update is part of **one thread**
- The entire conversation history is visible in linear order on **one page**
- No fragmentation across multiple views, tabs, or subthreads
- Users can see the full context by scrolling through the thread

Example flow:

```
Employee A (10:00): "We need approval to hire a new engineer"
Manager B (10:15): "What's the budget impact?" [Question in thread]
Employee A (10:30): "~$80k/year plus benefits" [Reply in same thread]
Manager C (11:00): "Approved from Finance" [Approval in same thread]
CEO (11:15): "Final sign-off from leadership" [Final approval in same thread]
→ Entire conversation visible on one page
```

No separate "approval" messages or "payment status" sections—everything flows through the same thread.

4. Access Control & Visibility

4.1 Room-Based Access

Each employee has membership in:

- 1 Primary branch room (mandatory)
- 1+ Department rooms (based on role)
- Founder room (if executive/founder)

4.2 Visibility Rules

Within a room:

- All tickets assigned to that room are visible to all members
- Tickets in other rooms are NOT visible
- Conversation threads within a ticket are fully visible to room members

Cross-room tickets:

- A ticket can be "assigned" to multiple rooms
- Appears in all assigned rooms' feeds
- Members of each room see it and can comment
- All comments remain in the same thread (one unified thread)

Example:

- Finance Manager creates ticket: "Q3 Budget Review"
- Assigns it to: [Finance Room] + [Operations Room]
- Finance team sees it in Finance Room
- Operations team sees it in Operations Room
- Both teams see the same thread with all comments mixed

4.3 Privacy

- Employees cannot see other rooms' tickets unless explicitly assigned
 - Each room is a privacy boundary—tickets stay scoped
 - Prevents information silos while maintaining control
-

5. Data Model

5.1 Core Entities

Employee

```
{
  id: UUID,
  name: string,
  email: string,
  branch_id: UUID,           // Primary branch
  department_ids: [UUID], // One or more
  created_at: timestamp
}
```

Room

```
{
  id: UUID,
  name: string,           // "Branch A", "Finance", "Founder"
  type: enum ["branch", "department", "founder"],
  members: [Employee],    // Derived from employee.branch_id /
  department_ids
  created_at: timestamp
}
```

Ticket

```
{
  id: UUID,
  title: string,
  description: string,
  creator_id: UUID,       // Employee who created it
  assigned_rooms: [UUID], // Which rooms this ticket appears in
  assigned_to: UUID,      // Which person/team is executing it
  status: enum ["open", "in_progress", "approved", "resolved"],
  priority: enum ["low", "medium", "high"],
  due_date: timestamp,
  created_at: timestamp,
  updated_at: timestamp
}
```

Message (comment/thread entry)

```
{
  id: UUID,
  ticket_id: UUID,        // Which ticket this belongs to
  author_id: UUID,        // Who wrote it
  content: string,        // The actual message
  type: enum ["comment", "approval", "status_update"],
  created_at: timestamp
}
```

5.2 Key Relationships

- **Employee → Room:** Many-to-many (employee in multiple rooms)
- **Room → Ticket:** One-to-many (room contains many tickets)

- **Ticket → Message:** One-to-many (ticket has many messages)
 - **Message → Author:** Many-to-one (many messages, same author)
-

6. Workflow Patterns

6.1 Simple Approval Flow

1. **Request phase** (Branch A Room)
 - Employee posts: "Need approval to purchase new equipment (\$5k)"
2. **Review phase** (same thread)
 - Manager asks: "What's the business justification?"
 - Employee responds: "Increases productivity by 30%"
3. **Approval phase** (same thread)
 - Finance approves: "Budget available, approved"
 - Requester: "Thank you, ordering now"
4. **Completion** (same thread)
 - Requester: "Equipment received, installation complete"
 - Thread resolved

All visible on one page, one room—no jumping between tabs.

6.2 Cross-Team Coordination

1. Operations creates ticket: "Onboarding new hire"
 2. Assigned to: [Operations Room] + [HR Room] + [Engineering Room]
 3. Operations posts: "New engineer starting Monday"
 4. HR posts: "I need their paperwork"
 5. Engineering posts: "Setting up dev environment"
 6. All three teams see the full conversation in their respective rooms
 7. Single thread, unified visibility
-

7. Features & Capabilities

7.1 Core Features

- **Room management:** Create, list, manage room membership
- **Ticket CRUD:** Create, read, update, delete tickets

- **Threading:** Unlimited comments per ticket
- **Cross-room assignment:** Assign tickets to multiple rooms
- **Status tracking:** Track ticket lifecycle (open → approved → resolved)
- **Search:** Find tickets by title, content, room, assignee
- **Notifications:** Alert relevant members of new tickets or comments (TBD)

7.2 Future Extensions

- **Approvals workflow:** Formal approval chains with sign-offs
 - **Templates:** Ticket templates for common workflows
 - **File attachments:** Upload documents, images
 - **Mentions:** @mention specific people
 - **Tags:** Categorize tickets (bug, feature request, etc.)
 - **Automation:** Rules for auto-routing or auto-assigning
-

8. Why Not a CRM?

CRM systems are built for **customer data and sales management**:

- Heavy emphasis on contact/lead/deal storage
- Designed around customer interactions
- Often bloated with features you don't need
- Tight coupling to sales workflows

This system is **communication-first**:

- No customer database
- No lead pipeline
- No deal stages
- Focused on internal task coordination and transparency
- Asynchronous, thread-based conversations
- Room-scoped for privacy and organization

A CRM would be overkill and would actually make collaboration **harder**—too many layers, wrong mental model.

9. Technical Considerations

9.1 Database

- Relational (PostgreSQL) for ACID compliance
- Tables: employees, rooms, room_members, tickets, messages
- Indexes on: ticket.assigned_rooms, ticket.status, message.ticket_id
- Soft deletes on sensitive data (tickets, messages)

9.2 API Design

Endpoints (REST/GraphQL)

```
POST /rooms                # Create room
GET /rooms/:id/tickets     # List tickets in room
POST /rooms/:id/tickets    # Create ticket in room
GET /tickets/:id          # Fetch full ticket + thread
POST /tickets/:id/messages # Add message to ticket
PATCH /tickets/:id        # Update ticket (status, assignee)
POST /tickets/:id/assign_rooms # Assign to additional rooms
```

9.3 Real-time Updates

- WebSocket or Server-Sent Events for live thread updates
- When someone posts a comment, all room members see it immediately
- Notification when you're mentioned

9.4 Security

- Role-based access control (RBAC): employee, manager, founder
- Row-level security: employees can only see tickets in their rooms
- Audit log: who created, modified, deleted what and when

10. Scaling Considerations

10.1 Current Scale

- ~200 employees
- ~3 branches
- ~5-6 departments

- Estimated 100-500 tickets per month
- ✓ Single database, simple app server

10.2 Future Scale (if needed)

- Millions of messages → archive old tickets
 - Real-time subscriptions → Redis pub/sub or dedicated WebSocket service
 - Search → Elasticsearch
 - Notifications → separate queue (Bull, RabbitMQ)
 - Read replicas for reporting
-

11. UI/UX Guidelines

11.1 Room View

- Left sidebar: List of rooms (Branch A, HR, Finance, Founder)
- Main area: Tickets in selected room (list with title, status, assignee)
- Click ticket → full thread in center panel

11.2 Ticket Detail View

- Ticket header: Title, status, assigned rooms, due date
- Thread panel: Chronological messages with author names and timestamps
- Comment box at bottom: Quick reply
- No collapsing—all messages visible

11.3 Ticket Creation

- Room selector (required): Which room(s) does this belong to?
 - Title + description
 - Assignee (optional, can be empty or assigned later)
 - Priority, due date (optional)
-

12. Success Metrics

- **Adoption:** % of employees using the system daily
- **Message velocity:** Avg messages per ticket (indicates engagement)
- **Resolution time:** Avg time from create → resolved

- **Cross-room efficiency:** % of tickets assigned to multiple rooms, avg response time
 - **Search usage:** % of lookups using search vs. scrolling
 - **User satisfaction:** NPS on ease of use, clarity of workflow
-

13. Comparison to Market Alternatives

See **Section below** for market analysis.

Implementation Roadmap

Phase 1 (MVP)

- Room management
- Ticket CRUD
- Threaded comments
- Basic search

Phase 2

- Cross-room assignment
- Status workflow
- Real-time notifications

Phase 3

- Approval chains
- Templates
- File attachments
- Advanced search

Phase 4

- Automation rules
 - Analytics dashboard
 - Mobile app
-